Indian Institute of Space Science and Technology
Thiruvananthapuram

Course on Python Programming, Data Processing & Visualization

(28 September - 09 October 2026)

Python Fundamentals - Part II

Input · Output · Type Conversion · Operators · Expressions

Dhevamathi C

Department of Avionics

Indian Institute of Space Science and Technology, Trivandrum

29 September 2026 · Day 2 - Session 2

Where We Are Now

What you already know

- What is a variable
- Data types: `int`, `float`, `str`, `bool`
- How to store values with `=`

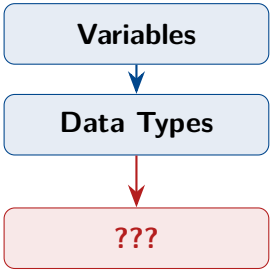
```
age = 25
name = "Ravi"
height = 175.5
is_soldier = True
```

Where We Are Now

What you already know

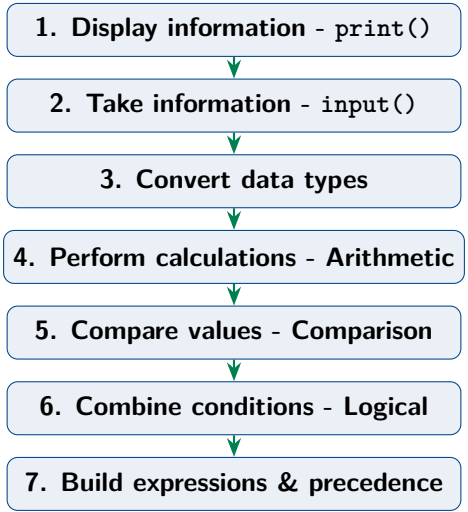
- What is a variable
- Data types: `int`, `float`, `str`, `bool`
- How to store values with =

```
age = 25
name = "Ravi"
height = 175.5
is_soldier = True
```



How do we make Python *interact* with us?

What We Will Learn Today



Section 1

Output Using `print()`

Talking to the Computer

The simplest Python instruction

```
print("Hello")
```

Talking to the Computer

The simplest Python instruction

```
print("Hello")
```

Output

```
Hello
```

Talking to the Computer

The simplest Python instruction

```
print("Hello")
```

Output

```
Hello
```



Talking to the Computer

The simplest Python instruction

```
print("Hello")
```

Output

```
Hello
```



Key idea: `print()` displays information on the screen.

What Can `print()` Display?

```
print("Hello") # a string
print(25) # a number (int)
print(3.14) # a number (float)
print(10 + 5) # a calculation
```

What Can `print()` Display?

```
print("Hello") # a string
print(25) # a number (int)
print(3.14) # a number (float)
print(10 + 5) # a calculation
```

Output

```
Hello
25
3.14
15
```

What Can `print()` Display?

```
print("Hello") # a string
print(25) # a number (int)
print(3.14) # a number (float)
print(10 + 5) # a calculation
```

Output

```
Hello
25
3.14
15
```

Notice: `print(10 + 5)` does *not* display `10 + 5`.
It **calculates first**, then displays the **result**.

Printing Variables

Since we already know how to create variables. . .

```
age = 25
name = "Ravi"
print(name)
print(age)
```

Printing Variables

Since we already know how to create variables. . .

```
age = 25
name = "Ravi"
print(name)
print(age)
```

Output

```
Ravi
25
```

Printing Variables

Since we already know how to create variables. . .

```
age = 25
name = "Ravi"
print(name)
print(age)
```

Output

```
Ravi
25
```

Remember

`print()` does not calculate by itself. It displays the **result** of whatever value or expression you give it.

Example: Personal Information

```
name = "Ravi"
age = 28
height = 170.5

print(name)
print(age)
print(height)
```

Example: Personal Information

```
name = "Ravi"
age = 28
height = 170.5

print(name)
print(age)
print(height)
```

Predict: What will Python display?

Example: Personal Information

```
name = "Ravi"
age = 28
height = 170.5

print(name)
print(age)
print(height)
```

Predict: What will Python display?

Output

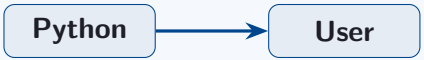
```
Ravi
28
170.5
```

Section 2

Input Using `input()`

From Output to Input

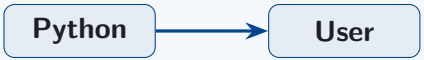
Previously - Output



Using `print()`

From Output to Input

Previously - Output



Using `print()`

Now - Input



Using `input()`

From Output to Input



`input()` lets the user give information to Python.

First input() Example

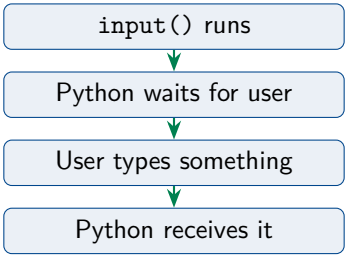
```
name = input("Enter your name: ")
print(name)
```

First input() Example

```
name = input("Enter your name: ")
print(name)
```

Interaction

```
Enter your name: Ravi
Ravi
```



The Most Important Rule About `input()`

`input()` always returns a string.

```
age = input("Enter your age: ")
```

The Most Important Rule About `input()`

`input()` always returns a string.

```
age = input("Enter your age: ")
```

If the user enters 25, Python receives:

What Python gets

"25"

(a **string**)

$\neq$

What you might expect

25

(a **number**)

Why Is This a Problem?

```
age = input("Enter age: ")
print(age + 5)
```

Why Is This a Problem?

```
age = input("Enter age: ")
print(age + 5)
```

Predict: Will this calculate age + 5?

Why Is This a Problem?

```
age = input("Enter age: ")
print(age + 5)
```

Predict: Will this calculate age + 5?

Error!

```
TypeError:  can only concatenate str (not "int") to str
```

Why Is This a Problem?

```
age = input("Enter age: ")
print(age + 5)
```

Predict: Will this calculate age + 5?

Error!

```
TypeError:  can only concatenate str (not "int") to str
```

Why? Because `"25"+ 5` is mixing a **string** with a **number**.

Solution: We need to **convert** the input!

Section 3

Type Conversion

What Is Type Conversion?

Definition

Changing a value from one data type to another.

What Is Type Conversion?

Definition

Changing a value from one data type to another.

Function	Converts to	Example	Result
`int()`	Integer	`int("25")`	25
`float()`	Decimal	`float("3.14")`	3.14
`str()`	Text	`str(25)`	"25"
`bool()`	True/False	`bool(1)`	True

What Is Type Conversion?

Definition

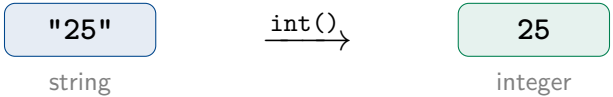
Changing a value from one data type to another.

Function	Converts to	Example	Result
`int()`	Integer	`int("25")`	25
`float()`	Decimal	`float("3.14")`	3.14
`str()`	Text	`str(25)`	"25"
`bool()`	True/False	`bool(1)`	True

Think of it as: translating between languages that Python understands.

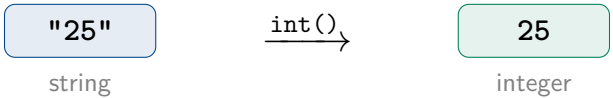
int() - Convert to Integer

```
age = int("25")
```



int() - Convert to Integer

```
age = int("25")
```



The most common pattern

```
age = int(input("Enter your age: "))
```



`float()` - Convert to Decimal

```
height = float(input("Enter your height: "))
```

Interaction

```
Enter your height: 172.5
```

`float()` - Convert to Decimal

```
height = float(input("Enter your height: "))
```

Interaction

```
Enter your height: 172.5
```

Python stores: `172.5` (a **float**, not a string)

`float()` - Convert to Decimal

```
height = float(input("Enter your height: "))
```

Interaction

```
Enter your height: 172.5
```

Python stores: `172.5` (a **float**, not a string)

When to use `int()` vs `float()`?

- **`int()`** - when the value is a whole number (age, count, marks)
- **`float()`** - when the value may have decimals (height, weight, price)

str() - Convert to Text

```
age = 25
message = "Age: " + str(age)
print(message)
```

str() - Convert to Text

```
age = 25
message = "Age: " + str(age)
print(message)
```

Output

```
Age: 25
```

str() - Convert to Text

```
age = 25
message = "Age: " + str(age)
print(message)
```

Output

```
Age: 25
```

Without str(), this would fail!

```
"Age: " + 25 # TypeError!
```

You cannot add a string and a number directly.

Conversion Cheat Sheet

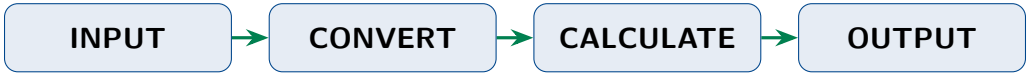
What I Have	What I Want	Use
`"25"`	`25`	`int()`
`"25.5"`	`25.5`	`float()`
`25`	`"25"`	`str()`
`1`	`True`	`bool()`

Conversion Cheat Sheet

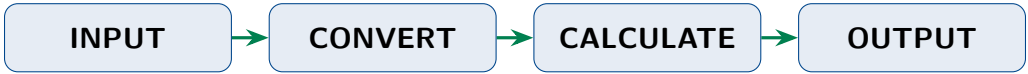
What I Have	What I Want	Use
`"25"`	`25`	`int()`
`"25.5"`	`25.5`	`float()`
`25`	`"25"`	`str()`
`1`	`True`	`bool()`

Tip: Keep this table in mind whenever you use `input()`.
Always convert before doing calculations!

The Complete Pattern



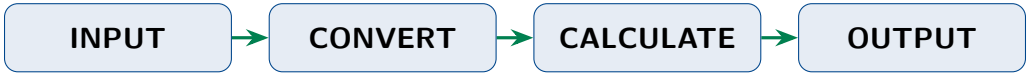
The Complete Pattern



Example: Calculate area of a rectangle

```
length = float(input("Enter length: "))
width = float(input("Enter width: "))
area = length * width
print("Area =", area)
```

The Complete Pattern



Example: Calculate area of a rectangle

```
length = float(input("Enter length: "))
width = float(input("Enter width: "))
area = length * width
print("Area =", area)
```

Interaction

```
Enter length: 10
Enter width: 5
Area = 50.0
```

Section 4

Operators

What Is an Operator?

```
a + b
```



What Is an Operator?

```
a + b
```

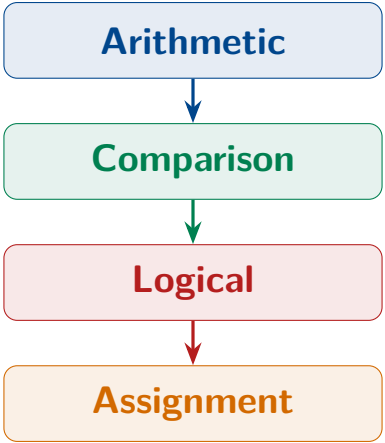


`a + b` is called an **expression**.

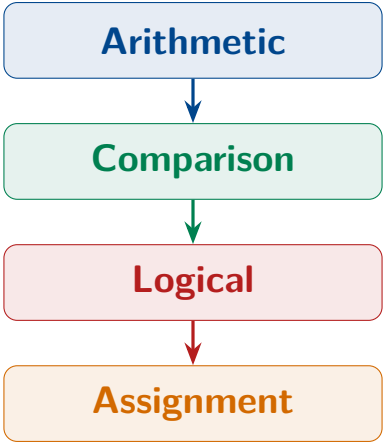
An **operator** performs an operation on values (operands).

An **expression** is something Python **evaluates** to produce a result.

Operators We Cover Today



Operators We Cover Today



Membership and identity operators (`in`, `is`) and bitwise operators are covered on other days.

Arithmetic Operators

Operator	Example	Meaning
+	`10 + 3`	Addition
-	`10 - 3`	Subtraction
*	`10 * 3`	Multiplication
/	`10 / 3`	Division
//	`10 // 3`	Floor division (no decimals)
%	`10 % 3`	Remainder (modulus)
**	`10 ** 3`	Power (exponentiation)

Basic Arithmetic in Action

```
a = 10
b = 3

print(a + b)
print(a - b)
print(a * b)
```

Basic Arithmetic in Action

```
a = 10
b = 3

print(a + b)
print(a - b)
print(a * b)
```

Predict the outputs!

Basic Arithmetic in Action

```
a = 10
b = 3

print(a + b)
print(a - b)
print(a * b)
```

Predict the outputs!

Output

```
13
7
30
```

/ vs // - Two Types of Division

/ - Regular Division

```
print(10 / 3)
```

```
3.3333333333333335
```

Gives the **full decimal** result.

/ vs // - Two Types of Division

/ - Regular Division

```
print(10 / 3)
```

```
3.3333333333333335
```

Gives the **full decimal** result.

// - Floor Division

```
print(10 // 3)
```

```
3
```

Removes the decimal part.
Only keeps the whole number.

```
10 / 3   ⟶  3.333...
10 // 3  ⟶  3
25 / 4   ⟶  6.25
25 // 4  ⟶  6
```

% - The Remainder Operator

$$17 \div 5$$

$$5 \times 3 = 15$$

$$17 - 15 = \mathbf{2} \quad \leftarrow \text{remainder}$$

```
print(17 % 5)
```

```
2
```

% - The Remainder Operator

$$17 \div 5$$

$$5 \times 3 = 15$$

$$17 - 15 = \mathbf{2} \quad \leftarrow \text{remainder}$$

```
print(17 % 5)
```

```
2
```

Predict: What is 25 % 4 ?

% - The Remainder Operator

$$17 \div 5$$

$$5 \times 3 = 15$$

$$17 - 15 = \mathbf{2} \quad \leftarrow \text{remainder}$$

```
print(17 % 5)
```

```
2
```

Predict: What is 25 % 4 ?

$25 \div 4 = 6$ remainder $\mathbf{1} \quad \Rightarrow \quad$ `25 % 4 = 1`

Why Is % Useful?

Example: Check if a number is even or odd

```
number = 8
print(number % 2)
```

```
0
```

Why Is % Useful?

Example: Check if a number is even or odd

```
number = 8
print(number % 2)
```

```
0
```

- If `number % 2` gives 0 $\Rightarrow$ the number is **even**
- If `number % 2` gives 1 $\Rightarrow$ the number is **odd**

Why Is % Useful?

Example: Check if a number is even or odd

```
number = 8
print(number % 2)
```

```
0
```

- If `number % 2` gives 0 $\Rightarrow$ the number is **even**
- If `number % 2` gives 1 $\Rightarrow$ the number is **odd**

Tomorrow you will learn how to use this result with `if`/`else` to make decisions!

** - The Power Operator

```
print(2 ** 3) # 2 x 2 x 2
print(5 ** 2) # 5 x 5
print(10 ** 0) # any number to the power 0
```

** - The Power Operator

```
print(2 ** 3) # 2 x 2 x 2
print(5 ** 2) # 5 x 5
print(10 ** 0) # any number to the power 0
```

Output

```
8
25
1
```

$$
\begin{aligned}
2 \text{ ** } 3 &= 2 \times 2 \times 2 &&= 8 \\
5 \text{ ** } 2 &= 5 \times 5 &&= 25 \\
10 \text{ ** } 0 & &&= 1
\end{aligned}
$$

Arithmetic Challenge

```
a = 20
b = 6

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
```

Arithmetic Challenge

```
a = 20
b = 6

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
```

Predict all six answers!

Arithmetic Challenge

```
a = 20
b = 6

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
```

Predict all six answers!

Answers

26 14 120 3.333... 3 2

A Shortcut You Will Use All the Time

Suppose we want to **add 5** to a variable and **store it back**:

```
score = 10
score = score + 5
print(score)
```

```
15
```

A Shortcut You Will Use All the Time

Suppose we want to **add 5** to a variable and **store it back**:

```
score = 10
score = score + 5
print(score)
```

```
15
```

Same result, shorter code

```
score = 10
score += 5
print(score)
```

```
15
```

Augmented Assignment Operators

These shortcuts are called **augmented assignment operators**.

Augmented Assignment Operators

These shortcuts are called **augmented assignment operators**.

Operator	Same As	Meaning
`x += 5`	`x = x + 5`	Add and assign
`x -= 5`	`x = x - 5`	Subtract and assign
`x *= 5`	`x = x * 5`	Multiply and assign
`x /= 5`	`x = x / 5`	Divide and assign
`x //= 5`	`x = x // 5`	Floor-divide and assign
`x %= 5`	`x = x % 5`	Remainder and assign
`x **= 5`	`x = x ** 5`	Power and assign

Augmented Assignment Operators

These shortcuts are called **augmented assignment operators**.

Operator	Same As	Meaning
`x += 5`	`x = x + 5`	Add and assign
`x -= 5`	`x = x - 5`	Subtract and assign
`x *= 5`	`x = x * 5`	Multiply and assign
`x /= 5`	`x = x / 5`	Divide and assign
`x //= 5`	`x = x // 5`	Floor-divide and assign
`x %= 5`	`x = x % 5`	Remainder and assign
`x **= 5`	`x = x ** 5`	Power and assign

Key idea: It updates the variable **using its own current value**.

Augmented Assignment in Action

```
total = 100
total += 20 # total = total + 20
print(total)
total -= 30 # total = total - 30
print(total)
total *= 2 # total = total * 2
print(total)
total //= 3 # total = total // 3
print(total)
```

Augmented Assignment in Action

```
total = 100
total += 20 # total = total + 20
print(total)
total -= 30 # total = total - 30
print(total)
total *= 2 # total = total * 2
print(total)
total //= 3 # total = total // 3
print(total)
```

Predict all four outputs!

Augmented Assignment in Action

```
total = 100
total += 20 # total = total + 20
print(total)
total -= 30 # total = total - 30
print(total)
total *= 2 # total = total * 2
print(total)
total //= 3 # total = total // 3
print(total)
```

Predict all four outputs!

Output

120 90 180 60

Why Use Augmented Assignment?

Without shortcut

```
count = count +
    1
```

Repeats the variable name twice

Why Use Augmented Assignment?

Without shortcut

```
count = count +
    1
```

Repeats the variable name twice

With shortcut

```
count += 1
```

Shorter, cleaner, less error-prone

Why Use Augmented Assignment?

Without shortcut

```
count = count +
    1
```

Repeats the variable name twice

With shortcut

```
count += 1
```

Shorter, cleaner, less error-prone

Very common use - a running counter

```
distance_travelled = 0
distance_travelled += 15 # after day 1
distance_travelled += 22 # after day 2
print(distance_travelled)
```

```
37
```

Section 5

Comparison Operators

From Calculation to Question

Arithmetic asks:

"What is the **result**?"

`10 + 5` $\Rightarrow$ `15`

From Calculation to Question

Arithmetic asks:

"What is the **result**?"

`10 + 5` $\Rightarrow$ `15`

Comparison asks:

"Is this **True** or **False**?"

`10 > 5` $\Rightarrow$ `True`

From Calculation to Question

Arithmetic asks:

"What is the **result**?"

`10 + 5` $\Rightarrow$ `15`

Comparison asks:

"Is this **True** or **False**?"

`10 > 5` $\Rightarrow$ `True`

Comparison operators don't calculate a number.
They answer a **yes/no question**.

Comparison Operators

Operator	Example	Meaning
==	5 == 5	Equal to
!=	5 != 3	Not equal to
>	5 > 3	Greater than
<	3 < 5	Less than
>=	5 >= 5	Greater than or equal to
<=	3 <= 5	Less than or equal to

Comparison Operators

Operator	Example	Meaning
==	5 == 5	Equal to
!=	5 != 3	Not equal to
>	5 > 3	Greater than
<	3 < 5	Less than
>=	5 >= 5	Greater than or equal to
<=	3 <= 5	Less than or equal to

The result is always `True` or `False`.

Predict the Results

```
x = 10

print(x == 10)
print(x != 10)
print(x > 5)
print(x < 5)
print(x >= 10)
print(x <= 9)
```

Predict the Results

```
x = 10

print(x == 10)
print(x != 10)
print(x > 5)
print(x < 5)
print(x >= 10)
print(x <= 9)
```

True or False?

Predict the Results

```
x = 10

print(x == 10)
print(x != 10)
print(x > 5)
print(x < 5)
print(x >= 10)
print(x <= 9)
```

True or False?

Answers

`True` `False` `True` `False` `True` `False`

= vs == - Very Important!

= is ASSIGNMENT

```
x = 10
```

Store 10 in `x`

= vs == - Very Important!

= is ASSIGNMENT

```
x = 10
```

Store 10 in x

== is COMPARISON

```
x == 10
```

Check if x equals 10

= ⟶ ASSIGN **== ⟶ COMPARE**

Section 6

Logical Operators

Why Do We Need Logical Operators?

Suppose we have **two conditions**:

```
marks >= 40 # condition 1
attendance >= 75 # condition 2
```

Why Do We Need Logical Operators?

Suppose we have **two conditions**:

```
marks >= 40 # condition 1
attendance >= 75 # condition 2
```

What if **both must be true** for a student to pass?

Why Do We Need Logical Operators?

Suppose we have **two conditions**:

```
marks >= 40 # condition 1
attendance >= 75 # condition 2
```

What if **both must be true** for a student to pass?

We combine them!

```
marks >= 40 and attendance >= 75
```

Why Do We Need Logical Operators?

Suppose we have **two conditions**:

```
marks >= 40 # condition 1
attendance >= 75 # condition 2
```

What if **both must be true** for a student to pass?

We combine them!

```
marks >= 40 and attendance >= 75
```

Logical operators let us **combine** or **modify** conditions.

The Three Logical Operators

Operator	Meaning	Example
`and`	Both must be True	`A and B`
`or`	At least one is True	`A or B`
`not`	Reverses True/False	`not A`

and **or** **not**

and - Both Must Be True

A	B	A and B
True	True	**True**
True	False	**False**
False	True	**False**
False	False	**False**

and - Both Must Be True

A	B	A and B
True	True	**True**
True	False	**False**
False	True	**False**
False	False	**False**

Example

```
marks = 65
attendance = 80
passed = marks >= 40 and attendance >= 75
print(passed)
```

```
True
```

`or` - At Least One Must Be True

A	B	A `or` B
True	True	**True**
True	False	**True**
False	True	**True**
False	False	**False**

or - At Least One Must Be True

A	B	A or B
True	True	**True**
True	False	**True**
False	True	**True**
False	False	**False**

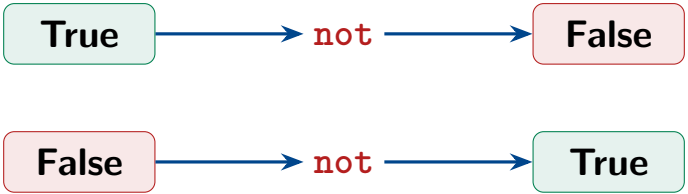
Example

```
score = 80
result = score < 40 or score > 75
print(result)
```

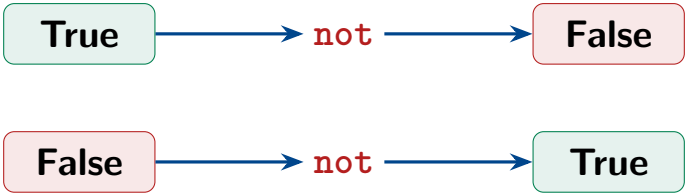
```
True
```

`score > 75` is True, so the whole expression is True.

`not` - Reverse True/False



not - Reverse True/False




```
print(not True) # False
print(not False) # True
print(not (10 > 5)) # not True -> False
```

Logical Operator Challenge

Given:

```
age = 25
```

Predict each result:

Logical Operator Challenge

Given:

```
age = 25
```

Predict each result:

1. `age >= 18 and age <= 60` ⇒ **True**
2. `age < 18 or age > 60` ⇒ **False**
3. `not(age >= 18)` ⇒ **False**

Logical Operator Challenge

Given:

```
age = 25
```

Predict each result:

1. `age >= 18 and age <= 60` ⇒ **True**
2. `age < 18 or age > 60` ⇒ **False**
3. `not(age >= 18)` ⇒ **False**

Preview

Tomorrow you will learn how to use these True/False results to **make decisions** using `if` and `else`!

Section 7

Expressions & Precedence

What Is an Expression?

An **expression** is anything Python **evaluates** to produce a value.

```
10 + 5 * 2
```

What Is an Expression?

An **expression** is anything Python **evaluates** to produce a value.

```
10 + 5 * 2
```

Is the answer 30 or 20?

What Is an Expression?

An **expression** is anything Python **evaluates** to produce a value.

```
10 + 5 * 2
```

Is the answer 30 or 20?

Answer: 20

Python does multiplication **before** addition:

$$5 \times 2 = 10 \quad \text{then} \quad 10 + 10 = \mathbf{20}$$

What Is an Expression?

An **expression** is anything Python **evaluates** to produce a value.

```
10 + 5 * 2
```

Is the answer 30 or 20?

Answer: 20

Python does multiplication **before** addition:

$$5 \times 2 = 10 \quad \text{then} \quad 10 + 10 = \mathbf{20}$$

This order of evaluation is called **operator precedence**.

Operator Precedence - Order of Evaluation

Priority	Operator
1 (Highest)	`()` Parentheses
2	`**` Exponentiation
3	`*` `/` `//` `%`
4	`+` `-`
5	`==` `!=` `>` `<` `>=` `<=`
6	`not`, `and`, `or`
7 (Lowest)	`=` `+=` `-=` `*=` `/=` `...` (assignment)

Operator Precedence - Order of Evaluation

Priority	Operator
1 (Highest)	`()` Parentheses
2	`**` Exponentiation
3	`*` `/` `//` `%`
4	`+` `-`
5	`==` `!=` `>` `<` `>=` `<=`
6	`not`, `and`, `or`
7 (Lowest)	`=` `+=` `-=` `*=` `/=` `...` (assignment)

Tip: When in doubt, use **parentheses** to make things clear!

Precedence Example

```
result = 10 + 5 * 2
```

Step 1: $5 \times 2 = 10$ (multiplication first)

Step 2: $10 + 10 = \mathbf{20}$ (then addition)

Precedence Example

```
result = 10 + 5 * 2
```

Step 1: $5 \times 2 = 10$ (multiplication first)

Step 2: $10 + 10 = \mathbf{20}$ (then addition)

More examples

```
print(2 + 3 * 4) # 2 + 12 = 14
print(2 ** 3 * 2) # 8 * 2 = 16
print(20 - 4 * 3 + 1) # 20 - 12 + 1 = 9
```

Parentheses Change the Order

Without parentheses

```
10 + 5 * 2
```

$$= 10 + 10 = \mathbf{20}$$

Parentheses Change the Order

Without parentheses

```
10 + 5 * 2
```

$= 10 + 10 = \mathbf{20}$

With parentheses

```
(10 + 5) * 2
```

$= 15 \times 2 = \mathbf{30}$

Parentheses Change the Order

Without parentheses

```
10 + 5 * 2
```

$= 10 + 10 = \mathbf{20}$

With parentheses

```
(10 + 5) * 2
```

$= 15 \times 2 = \mathbf{30}$

Best Practice

Always use **parentheses** when you want to make your intended calculation order obvious.

This is especially important in formulas like:

```
perimeter = 2 * (length + width)
```

Section 8

Putting It All Together

Example 1: How Old Are You?

```
name = input("Enter your name: ")
birth_year = int(input("Enter birth year: "))

age = 2026 - birth_year

print(name, "is", age, "years old")
print("You have lived", age * 365, "days!")
```

Example 1: How Old Are You?

```
name = input("Enter your name: ")
birth_year = int(input("Enter birth year: "))

age = 2026 - birth_year

print(name, "is", age, "years old")
print("You have lived", age * 365, "days!")
```

Interaction

```
Enter your name: Ravi
Enter birth year: 1990
Ravi is 36 years old
You have lived 13140 days!
```

Example 1: How Old Are You?

```
name = input("Enter your name: ")
birth_year = int(input("Enter birth year: "))

age = 2026 - birth_year

print(name, "is", age, "years old")
print("You have lived", age * 365, "days!")
```

Interaction

```
Enter your name: Ravi
Enter birth year: 1990
Ravi is 36 years old
You have lived 13140 days!
```

Concepts used: `input()`, `int()`, `-`, `*`, `print()` with commas

Example 2: Predict the Output!

What will Python print?

```
x = 10
y = 3
print(x + y * 2)
print((x + y) * 2)
print(x > y and y > 1)
print(x // y, x % y)
```

Example 2: Predict the Output!

What will Python print?

```
x = 10
y = 3
print(x + y * 2)
print((x + y) * 2)
print(x > y and y > 1)
print(x // y, x % y)
```

Answers

```
16 # multiplication first: 3*2=6, then 10+6
26 # brackets first: 10+3=13, then 13*2
True # 10>3 is True, 3>1 is True, True and True
3 1 # 10//3 = 3 (quotient), 10%3 = 1 (remainder)
```

Example 2: Predict the Output!

What will Python print?

```
x = 10
y = 3
print(x + y * 2)
print((x + y) * 2)
print(x > y and y > 1)
print(x // y, x % y)
```

Answers

```
16 # multiplication first: 3*2=6, then 10+6
26 # brackets first: 10+3=13, then 13*2
True # 10>3 is True, 3>1 is True, True and True
3 1 # 10//3 = 3 (quotient), 10%3 = 1 (remainder)
```

Key takeaway: Brackets change the order of operations!

Example 3: Even or Odd?

```
number = int(input("Enter a number: "))
remainder = number % 2
is_even = remainder == 0
print("Number:", number)
print("Remainder when divided by 2:", remainder)
print("Is it even?", is_even)
```

Example 3: Even or Odd?

```
number = int(input("Enter a number: "))
remainder = number % 2
is_even = remainder == 0
print("Number:", number)
print("Remainder when divided by 2:", remainder)
print("Is it even?", is_even)
```

Interaction

```
Enter a number: 7
Number: 7
Remainder when divided by 2: 1
Is it even? False
```

Example 3: Even or Odd?

```
number = int(input("Enter a number: "))
remainder = number % 2
is_even = remainder == 0
print("Number:", number)
print("Remainder when divided by 2:", remainder)
print("Is it even?", is_even)
```

Interaction

```
Enter a number: 7
Number: 7
Remainder when divided by 2: 1
Is it even? False
```

Concepts used: `int()`, modulo %, comparison ==, boolean result

Final Challenge: Speed Calculator

```
distance = float(input("Enter distance: "))
time = float(input("Enter time: "))
speed = distance / time
print("Speed =", speed)
```

Final Challenge: Speed Calculator

```
distance = float(input("Enter distance: "))
time = float(input("Enter time: "))
speed = distance / time
print("Speed =", speed)
```

Identify in this program:

- ✓ `input()` - taking data
- ✓ `float()` - type conversion
- ✓ Variables - `distance`, `time`, `speed`
- ✓ `/` - arithmetic operator
- ✓ `=` - assignment
- ✓ `print()` - output
- ✓ Expression: `distance / time`

Final Challenge: Speed Calculator

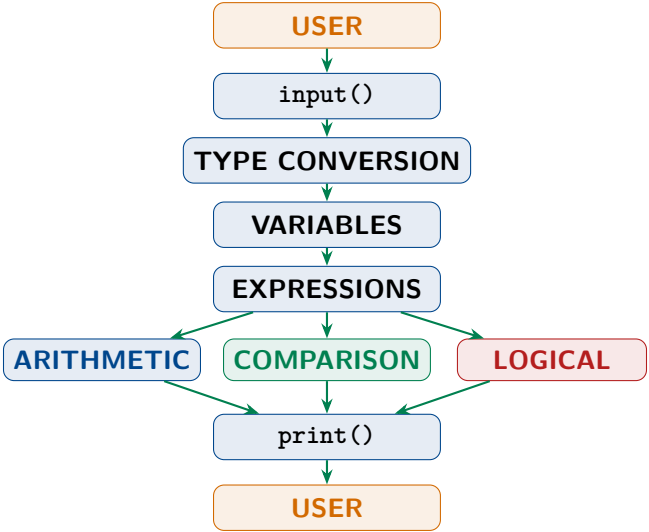
```
distance = float(input("Enter distance: "))
time = float(input("Enter time: "))
speed = distance / time
print("Speed =", speed)
```

Identify in this program:

- ✓ `input()` - taking data
- ✓ `float()` - type conversion
- ✓ Variables - `distance`, `time`, `speed`
- ✓ `/` - arithmetic operator
- ✓ `=` - assignment
- ✓ `print()` - output
- ✓ Expression: `distance / time`

Every concept from today's session is in this one program!

The Complete Picture



Thank You!

Questions?

Course on Python Programming, Data Processing & Visualization

IIST Trivandrum · 29 September 2026